Sentiment Analysis of Movie Reviews: Can a Model Understand "Not"?

CSC 425 AI Term Project, Final Report
Obaid Alnaqbi (solo project)

Project Summary

Problem. It was my objective to create a system that classifies movie reviews as positive or negative, and one question I asked myself was: can such a system recognize how the word "not" changes the meaning of a sentence?

Approach. TF-IDF features with Logistic Regression and Naive Bayes, using the IMDb Large Movie Review Dataset. I compared single words to word pairs, added a classic negation-marking step ("not boring" becomes "not not_boring"), tuned and calibrated the model, and tested it on IMDb, hand-written challenge sentences, single movie-review sentences and product reviews.

Major results. The final system achieves 90.7% accuracy on the 25,000-review IMDb test set (up from 85.6% for my first baseline), with confidence scores that are well calibrated (expected calibration error 0.01). On Amazon product reviews accuracy falls to 80.1%, and on single Rotten Tomatoes sentences it is 77.4%.

Most interesting finding. High accuracy doesn't necessarily imply understanding of negation. Even the final model gets only 4 of my 10 "not" sentence pairs fully right, and adding more training data does not help. The reason is in the data: on IMDb, "not bad" occurs more often in negative reviews than in positive ones (only 33% positive), so a model trained on the ratings of whole reviews learns that "not bad" indicates a bad review. Negation marking helps reliably, but only where reviewers use a negated phrase consistently.

1. Introduction

Sentiment analysis is the problem of deciding whether a text expresses a positive or negative opinion. It's something I do often without even thinking about it: before I see a movie, I check the reviews to find out what people thought of it. Companies do the same on a much larger scale, for instance to monitor how customers feel about a product, so they need systems that can automatically classify thousands of reviews.

Sentiment classification of movie reviews has been studied for over twenty years. Simple machine-learning models with word features were found to be effective (Pang, Lee and Vaithyanathan, 2002), and the IMDb dataset (Maas et al., 2011) became a standard benchmark. Since the basic task has been thoroughly researched, my instructor recommended adding one focused question to make the project my own. I chose negation. "The movie was enjoyable" and "The movie was not enjoyable" differ by only one word but are opposite in meaning, and bag-of-words models do not see word order, so it is not obvious that they can handle this.

The goals of the project were:

1. create a working sentiment classifier and a simple demo that anyone can type a review into;
2. test it on real reviews, using a held-out test set that is only used at the end;
3. check directly whether it has learnt the meaning of "not", using sentence pairs that I wrote before training;

4. try to improve the way negation is handled, and explain what works, what does not, and why.

My overall strategy was to keep it simple and to understand each step before adding anything. I started with a TF-IDF baseline (Milestone 2), then tuned it and tested negation marking, more data and several splits (Milestone 3), and finally calibrated the confidence scores, tested on two new datasets and built the demo (Milestone 4). I deliberately did not move to a neural network, as a small experiment that I can explain completely was more useful for answering my question.

2. Dataset and Problem Environment

Data sources

Dataset	Size used	Labels	Role
IMDb Large Movie Review Dataset (Maas et al., 2011)	25,000 train + 25,000 test	review-level, from star rating	training, validation, final test
Negation challenge pairs (my own)	10 pairs (20 sentences)	sentence-level	analysis only
Mixed-opinion and sarcasm sentences (my own)	10 + 10 sentences	sentence-level	analysis only
Rotten Tomatoes sentences (Pang & Lee, 2005)	1,066 (test split)	sentence-level	transfer test
Amazon product reviews, English (Keung et al., 2020)	4,000 (test split, no 3-star)	review-level, from star rating	domain transfer test

Table 1. Datasets used in the project.

IMDb has 50,000 reviews, half positive and half negative. Reviews rated 4/10 or lower are labeled negative and 7/10 or higher positive, and there are no neutral reviews. Reviews are about 230 words long on average (median about 175) and up to almost 2,000 words, and 59% of them contain the word "not". I downloaded the official splits from the Hugging Face mirror (stanfordnlp/imdb), since the Stanford server was not reachable. The Amazon reviews are rated 1 to 5 stars; I used the same rule as IMDb (1 to 2 stars negative, 4 to 5 stars positive) and excluded 3-star reviews.

Each of the three hand-written sets was written before any model was run on it, so I was not able to adjust the sets based on the predictions. There are 10 negation pairs; in 8 pairs a positive opinion becomes negative (e.g. "I would recommend this movie." vs "I would not recommend this movie.") and in 2 pairs a negative opinion becomes positive (e.g. "The film was boring." vs "The film was not boring."). All ten sarcasm sentences are negative.

Preprocessing and splits

Cleaning is deliberately light: remove the
 HTML tags found in many reviews, lowercase, and collapse whitespace. I never remove stop words, as the common stop-word lists contain "no", "not" and "never", which would remove my signal word. Tokenization is done by scikit-learn's TfidfVectorizer. For model development, I randomly sampled a balanced set of 2,000 positive and 2,000 negative training reviews (seed 42) and split it 80/20 into 3,200 training and 800 validation reviews. The bigger experiments add more training reviews from the remaining 24,200, but keep the same 800 validation reviews. No decisions were made using the official test set.

Limits and difficulties with the data

- **Review-level labels.** IMDb labels are for the entire review, not individual sentences. For the negation question (Section 6) this proved to be the most important limitation.
- **Label noise.** Some of the reviews look mislabeled. One review labeled negative says "This is definitely one of the best Kung fu movies in the history of Cinema".
- **Only clear opinions.** IMDb has only strongly positive or negative ratings, so the model never sees neutral or mixed ratings, and all reviews are in English and about movies.

3. Methodology

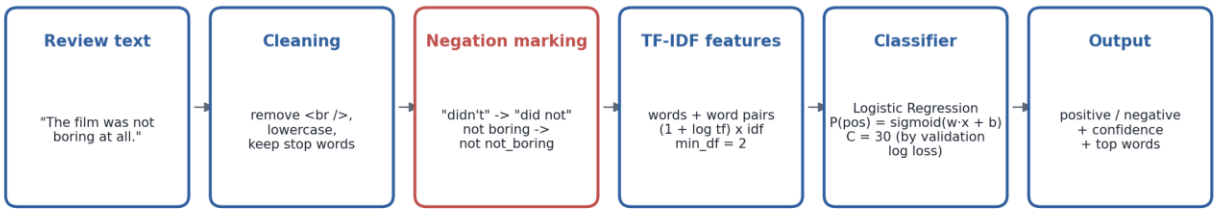


Figure 1. The final system. Each step is explained below.

TF-IDF features

Each review turns into a vector with one value per vocabulary term t. The value is tfidf(t, d) = (1 + log tf(t, d)) x idf(t), where tf is how often the term appears in the review and idf(t) = log((1 + N) / (1 + df(t))) + 1 is high when the term t is rare in the N training reviews. The vectors are then scaled so that each has length 1. The log damps reviews that repeat a word many times. Only terms that appear in at least two reviews are kept (min_df = 2). The terms can be single words (ngram_range = (1, 1)), or single words plus pairs of adjacent words (ngram_range = (1, 2)), so that "not good" can be a term on its own.

Classifiers

Logistic Regression learns one weight w_t per term and predicts P(positive | x) = 1 / (1 + exp(-(w · x + b))). Training minimizes the log loss plus an L2 penalty on the weights, and C sets the trade-off (small C = strong regularization). This makes it simple to inspect: I can list which words push a prediction one way and which push it the other, which the demo shows. Multinomial Naive Bayes was the second baseline. It estimates P(term | class) from smoothed counts (smoothing alpha) and multiplies these estimates, assuming the terms are independent.

Negation marking

I follow the approach of Das and Chen (2001) and Pang et al. (2002), with a preprocessing step that makes a word that follows a negation different from the same word without one:

1. Expand contractions: "didn't" becomes "did not", "won't" becomes "will not". Otherwise the tokenizer turns "didn't" into "didn" and the negation is lost.
2. Go through the tokens. After a negation word (not, no, never, nothing, nobody, none, neither, nor, cannot, without), put not_ in front of every word that follows.
3. Stop marking at punctuation or at a contrast word (but, however, although, though, yet), so "not great, but fun" does not turn "fun" into "not_fun".

For example, "the film was not boring at all." becomes "the film was not not_boring not_at not_all .". Now "boring" and "not_boring" have separate weights.

Calibration

The demo displays a confidence score, so it should be honest: of the predictions made with 80% confidence, about 80% should be correct, of those made with 90% confidence about 90%, and so on. I measure this with the expected calibration error (ECE): predictions are grouped into confidence bins, and ECE is the average gap between confidence and accuracy in each bin, weighted by bin size. In Milestone 3 I chose C by F1, which resulted in C = 1000 and an overconfident model. For the final system I chose C by validation log loss instead, which rewards predictions that are both correct and honest.

Key implementation decisions

- **Stop words preserved,** so the classifier can see the words "not", "no" and "never".
- **Test set used only for final evaluation,** so improvements cannot be explained by tuning on the test data.
- **Logistic Regression in the final system,** as it is well calibrated after tuning, fast, and fully explainable.

4. Experimental Setup

All experiments are run in Python 3.13, scikit-learn 1.7, pandas and matplotlib, with random seed 42, on a laptop CPU in less than 10 minutes in total. There is one script for each milestone that reproduces all the numbers of that milestone (src/run_milestone2.py, run_milestone3.py, run_milestone4.py).

Experiment	What is compared	Evaluated on
Baseline (M2)	LogReg vs Naive Bayes; words vs words+pairs; default settings	validation, negation pairs
Tuning (M3)	C in 0.1 to 1000, alpha in 0.01 to 1	validation F1
Negation marking (M3)	4 feature settings x 2 classifiers, tuned	validation, all challenge sets
Stability (M3)	LogReg settings on 5 other random 3,200/800 splits	mean and spread
Learning curve (M3)	3,200 to 24,200 training reviews	validation, negation pairs
Calibration (M4)	C in 0.3 to 1000 for the final setting	validation log loss, ECE
Final evaluation (M3, M4)	M2 baseline, M3 model, final system	IMDb test, Rotten Tomatoes, Amazon

Table 2. Experiments. M2/M3/M4 = milestone in which each was run.

Baseline: Logistic Regression using words + pairs and default settings, trained on 3,200 reviews (the best model from Milestone 2). A majority-class predictor (50%) is the floor. Metrics: accuracy, precision, recall and F1 (positive class) on real reviews; ECE and log loss for confidence; and for the challenge sets, the number of sentences right and the number of negation pairs where both sentences are right. I evaluated two models on the test set, both selected before their test score was known: the Milestone 3 model (selected by a fixed rule on validation F1) and the final system (selected using the stability check and calibration on the validation set). I report both openly.

5. Results

5.1 From the baseline to the final system on IMDb

Model	Train reviews	Accuracy	Precision	Recall	F1	ECE
M2 baseline	3,200	0.856	0.852	0.862	0.857	0.214
M3 model	25,000	0.905	0.906	0.904	0.905	0.048
Final system	**25,000**	**0.907**	**0.907**	**0.907**	**0.907**	**0.010**

Table 3. Official IMDb test set (25,000 reviews). M2 baseline: words+pairs, C = 1. M3 model: words+pairs, C = 1000. Final system: words+pairs+neg, C = 30.

The final system makes about 35% fewer errors than the first baseline (9.3% error as opposed to 14.4%). This is mainly due to more training data and tuning; negation marking adds only a little accuracy (90.5% to 90.7%). The bigger difference between the last two rows is calibration: the Milestone 3 model with C = 1000 had an ECE of 0.048, the final system 0.010. Errors are balanced across the two classes.

5.2 Tuning, negation marking and stability

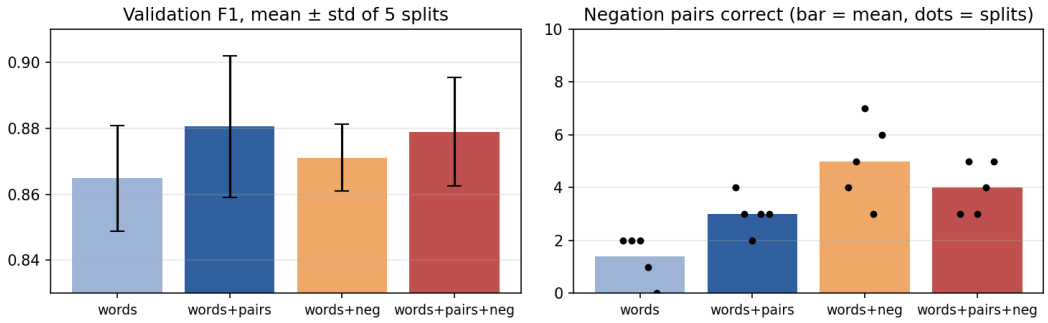


Figure 2. Logistic Regression on 5 random 3,200/800 splits. Left: validation F1. Right: negation pairs.

For each setting (e.g. words + pairs), tuning C increased validation F1 by about two points (0.881 to 0.902 for words + pairs). On a single split, negation marking appeared to lower F1 slightly. The stability check over five splits showed that the F1 differences between the four settings (0.865 to 0.881) were smaller than the split-to-split variation (standard deviation 0.010 to 0.021), so on real reviews they are essentially tied. The negation effect was clear, however: with single words, negation marking raised the average number of correct pairs from 1.4 to 5.0 and won on all five splits. The gain was smaller with word pairs (3.0 to 4.0).

5.3 More data

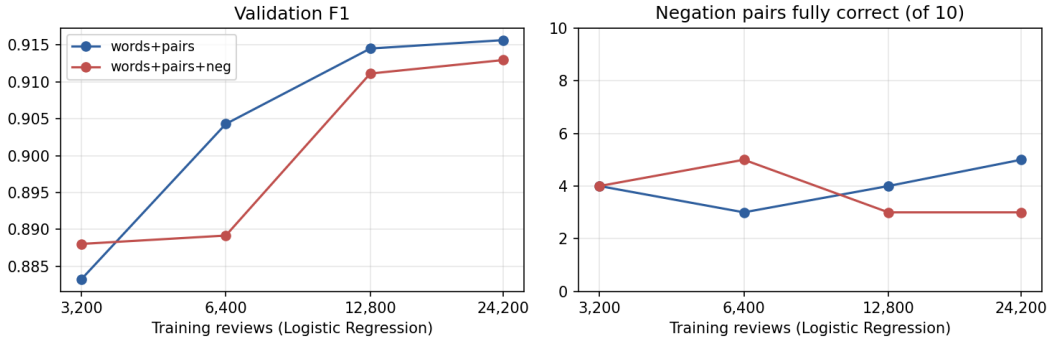


Figure 3. Logistic Regression with more training reviews (same 800 validation reviews).

When the number of training reviews increased from 3,200 to 24,200, validation F1 rose from 0.883 to 0.916, but the number of negation pairs correctly identified stayed in the range 3 to 5, with or without marking. More data made the model better on real reviews, but not on negation.

5.4 Calibration

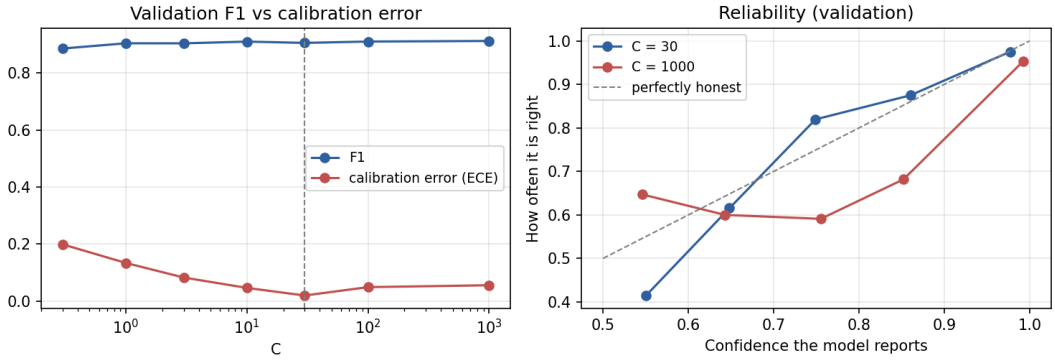


Figure 4. Left: F1 is almost level across C, while the calibration error has a clear minimum at C = 30. Right: reliability on validation data (closer to the diagonal = more honest).

F1 hardly changes between C = 1 and C = 1000 (0.905 to 0.913), but the calibration error is lowest at C = 30 (0.019 on validation, 0.010 on test). With C = 1000 the model was overconfident: its predictions made with about 85% confidence were right only about 68% of the time. Selecting C by log loss instead of F1 gave almost the same accuracy with much more reliable confidence.

5.5 Challenge sentences

Model	Negation pairs fully correct	Mixed opinions	Sarcasm
M2 baseline	3/10	8/10	4/10
M3 model	4/10	9/10	7/10
Final system	4/10	9/10	7/10

Table 4. Hand-written challenge sets.

Original	Final system	With "not"	Final system
The movie was enjoyable.	positive (0.93)	The movie was not enjoyable.	negative (0.95)
I would recommend this movie.	positive (1.00)	I would not recommend this movie.	negative (1.00)
The acting was good.	**negative (0.50) ***	The acting was not good.	negative (1.00)
This film is worth watching.	positive (0.98)	This film is not worth watching.	negative (1.00)
I liked the ending.	positive (0.99)	I did not like the ending.	negative (0.64)
The plot made sense.	**negative (0.68) ***	The plot did not make sense.	negative (0.99)
It was a great movie.	positive (1.00)	It was not a great movie.	**positive (0.68) ***
The jokes were funny.	**negative (0.74) ***	The jokes were not funny.	negative (1.00)
The film was boring.	negative (1.00)	The film was not boring.	**negative (0.84) ***
The sequel was bad.	negative (1.00)	The sequel was not bad.	**negative (0.98) ***

*Table 5. Final system on each negation pair, with confidence. * = wrong.*

The final system is correct on 7 of the 10 negated sentences, but on only 4 of the 10 pairs, partly because it now misreads some simple positive sentences ("The jokes were funny." is predicted negative). The predictions for both "not boring" and "not bad" remain negative. Mixed opinions are handled well (9/10). Sarcasm reaches 7/10, but since all sarcasm sentences are negative, this partly reflects how often the model predicts negative for short sentences.

5.6 New domains and sentence-level labels

Dataset	Model	Accuracy	F1	ECE
Rotten Tomatoes sentences (1,066)	M2 baseline	0.631	0.720	0.090
Rotten Tomatoes sentences (1,066)	M3 model	0.764	0.782	0.122
Rotten Tomatoes sentences (1,066)	**Final system**	**0.774**	**0.778**	**0.027**
Amazon product reviews (4,000)	M2 baseline	0.674	0.736	0.083
Amazon product reviews (4,000)	M3 model	0.783	0.794	0.125
Amazon product reviews (4,000)	**Final system**	**0.801**	**0.801**	**0.025**

Table 6. Models trained on IMDb, tested on new data without retraining.

Accuracy dropped by about 10 points on product reviews (80.1%) and 13 points on single sentences (77.4%). For products this is mainly a vocabulary issue: words like "stopped working" for a blender are rarely found in movie reviews. Single sentences were difficult because they contain few words. The final system was the best model on both datasets and also kept good calibration (ECE 0.03). On Rotten Tomatoes, sentences with a negation word were not more difficult than other sentences: the final system got 79.8% of them right, compared with 76.6% of the others. Negation marking helped slightly more on sentences with negation (+1.9 points over the M3 model) than on sentences without negation (+0.7).

5.7 Demo

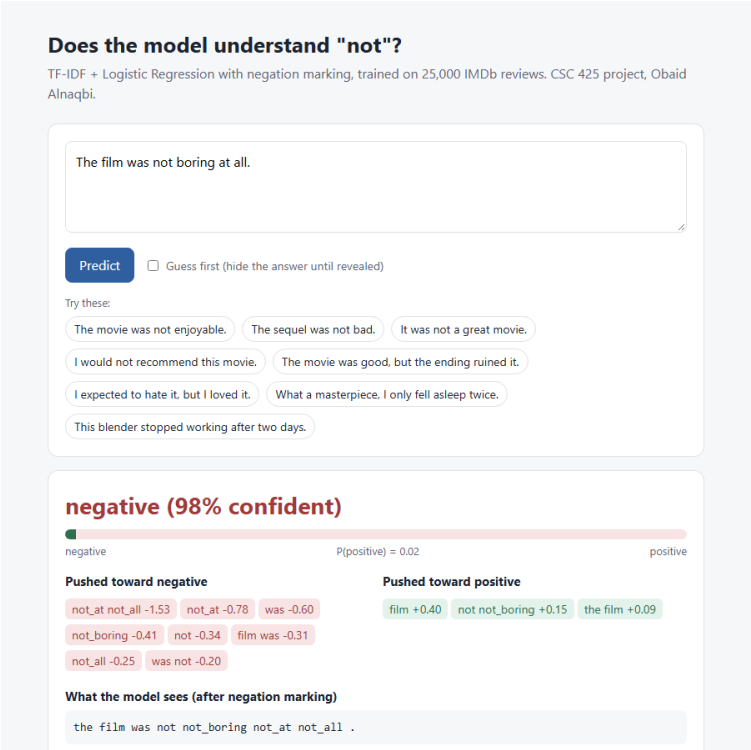


Figure 5. The web demo (python src/demo_app.py) shows the prediction, the confidence, the words that pushed it positive or negative, and the text after negation marking. It also has a "guess first" mode for the class.

The demo provides a view of how the model came to its decision. For the sentence "The film was not boring at all." the model predicts negative with high confidence (98%), and the explanation shows why: the phrase "not_at not_all" is the strongest negative signal, as it is found in many reviews such as "not funny at all", and "not_boring" itself still has a negative weight.

6. Discussion and Conclusions

Successes

- **A strong, simple classifier.** More training data, tuning and calibration increased accuracy by 5.1 points (85.6% to 90.7%) and provided honest confidence levels, which is close to what simple bag-of-words models typically achieve on IMDb.
- **Negation marking.** The negation rule improved the negation pairs across all 5 splits without hurting accuracy.
- **Repeating experiments.** Retesting on 5 splits meant I didn't come to an incorrect conclusion based on one lucky split.

Failures, and why

The model never learnt to flip "not bad" and "not boring", and additional data did not help. Upon inspection of the data, it was clear why. Of the 274 IMDb training reviews containing "not bad", only 33% are positive, whereas "not great" is 52% positive and "not good" 24%. People often write "the acting was not bad, but..." in reviews they rate poorly. A model trained on review ratings can only learn which phrases go with which rating; it cannot understand the meaning of a

phrase in its sentence. So for "The sequel was not bad." the model is actually an accurate reflection of IMDb: "not bad" really does predict a bad review there. My challenge set asks a question about the sentence, which cannot be answered using review-level labels.

Another reason is my negation rule. Because it marks "until punctuation", the negation sometimes applies to a long section of text. For example, in "do not miss this artistic noir film from one of the great film directors", "great" becomes not_great inside a very positive review. As a result, not_great ended up with a positive weight (+4.3). Real phrases also had an impact: not_like has a strong positive weight because of "I have never seen anything like it".

Interesting findings

- **Common positive words are not positive on IMDb.** "good" has almost no weight and "funny" is negative, because they are used in negative contexts as often as positive ones ("could have been good", "unintentionally funny").
- **Some errors are not the model's fault.** Several of the model's most confident test errors look mislabeled or sarcastic ("Master P's acting skills make you actually believe he is Italian!").
- **Short sentences are harder.** "I expected to hate it, but I loved it." was easy for the model, but "The jokes were funny." was not.

Limitations

All conclusions about negation are based on 10 hand-written pairs, which is a small amount of data; I tested on 5 splits, but that is still small. The system only works well on English movie reviews. A bag-of-words model cannot take word order into account beyond neighbouring pairs, so long-range negation and sarcasm are out of reach.

Future improvements

- **Sentence-level training data.** Train or fine-tune on sentence-level labels (for example the Stanford Sentiment Treebank, which labels every phrase) so the model can learn what "not bad" means locally.
- **Better negation scope.** Mark only 1 to 2 words after a negation, or use a dependency parser to find the negated word.
- **A model that reads order.** A small pretrained model such as DistilBERT reads word order and would be a natural next comparison, now that the baseline and the negation test set exist.
- **Larger test set.** Write a larger challenge set (50 to 100 pairs) to measure negation more precisely.

The main lesson I take from this project is that a good accuracy number can hide what a model does not understand, and that looking at the training data is often the fastest way to find out why.

7. Ethical Considerations

Sentiment classifiers can save people a lot of time, for example by summarizing thousands of customer reviews, and a transparent linear model like this one lets users see why it made each decision. But there are real risks:

- **Reliability outside its domain.** The model is 90.7% accurate on IMDb but only about 77 to 80% on other text, and it misreads negation and sarcasm. If it were used to make decisions

about people or products (flagging "negative" employees, removing reviews), these errors would fall unevenly on people who write with irony, negation or unusual vocabulary.

- **Bias and fairness.** Writing style varies by dialect, age, culture and native language. A model trained on English IMDb reviews may systematically misjudge non-native speakers or dialects it rarely saw. I did not measure this, which is itself a limitation.
- **Privacy.** IMDb reviews are public and the dataset is anonymized, and my demo does not store anything a user types. A deployed system that analyses customers' messages would need consent and clear limits on what is stored.
- **Misuse.** Sentiment analysis can be used to monitor employees or citizens, or to manipulate opinion by measuring which messages work. These uses deserve caution.
- **Transparency and honest confidence.** A confident but wrong answer is worse than an uncertain one. This is why I calibrated the model so that its confidence matches its real accuracy, and why the demo shows which words drove each prediction instead of only a label.

Responsible deployment would mean testing on the actual target text first, reporting accuracy for different groups of writers, keeping a human in the loop for any decision that affects people, and clearly stating that the model does not understand negation or sarcasm reliably.

8. Individual Contributions

This was a solo project. I (Obaid Alnaqbi) was responsible for all parts: choosing the topic and focus question, writing the challenge sentences, data preparation, model development, experiments, analysis, the demo, the reports and the presentation.

References

[1] Das, S., & Chen, M. (2001). Yahoo! for Amazon: Extracting market sentiment from stock message boards. Proceedings of the Asia Pacific Finance Association Annual Conference.

[2] Guo, C., Pleiss, G., Sun, Y., & Weinberger, K. Q. (2017). On calibration of modern neural networks. Proceedings of ICML 2017, 1321-1330.

[3] Keung, P., Lu, Y., Szarvas, G., & Smith, N. A. (2020). The Multilingual Amazon Reviews Corpus. Proceedings of EMNLP 2020, 4563-4568. (Data: huggingface.co/datasets/mteb/amazon_reviews_multi)

[4] Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., & Potts, C. (2011). Learning word vectors for sentiment analysis. Proceedings of ACL 2011, 142-150. (Data: ai.stanford.edu/~amaas/data/sentiment/, mirror huggingface.co/datasets/stanfordnlp/imdb)

[5] Pang, B., & Lee, L. (2005). Seeing stars: Exploiting class relationships for sentiment categorization with respect to rating scales. Proceedings of ACL 2005, 115-124. (Data: huggingface.co/datasets/cornell-movie-review-data/rotten_tomatoes)

[6] Pang, B., Lee, L., & Vaithyanathan, S. (2002). Thumbs up? Sentiment classification using machine learning techniques. Proceedings of EMNLP 2002, 79-86.

[7] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

[8] Socher, R., et al. (2013). Recursive deep models for semantic compositionality over a sentiment treebank. Proceedings of EMNLP 2013, 1631-1642.